

CAD fot VLSI PA3 : Analog Placement

Student : 卓祐晟

Student ID : 111521051

Date : 2024/5/26

Course : CAD for VLSI Design

Supervisor : Professor 陳聿廣

Outline

- CAD fot VLSI PA3 : Analog Placement
 - Outline
 - 1. Assignment Requirement
 - 2. How to Compile and Execute and Completion
 - 4. Program Details (Algorithms and Data Structures).
 - Read input file to build my_nets, my_blocks, my_symmetrys;
 - B* tree
 - Preorder Traversal
 - Y Contour
 - Symmetry Placement
 - Calculate HPWL
 - 5. Perturbation Strategy and Operations & Cost Function
 - 6. My Suggestion

1. Assignment Requirement

You are asked to implement your own analog placer. We covered various analog placement algorithms in class, including B*-tree and Sequence Pair. Choose a feasible algorithm to complete your programming assignment.

Input: SPICE netlist file, device group file, technology file, symmetry constraint file

Intermediate file: Building block file

Output: Placement result file

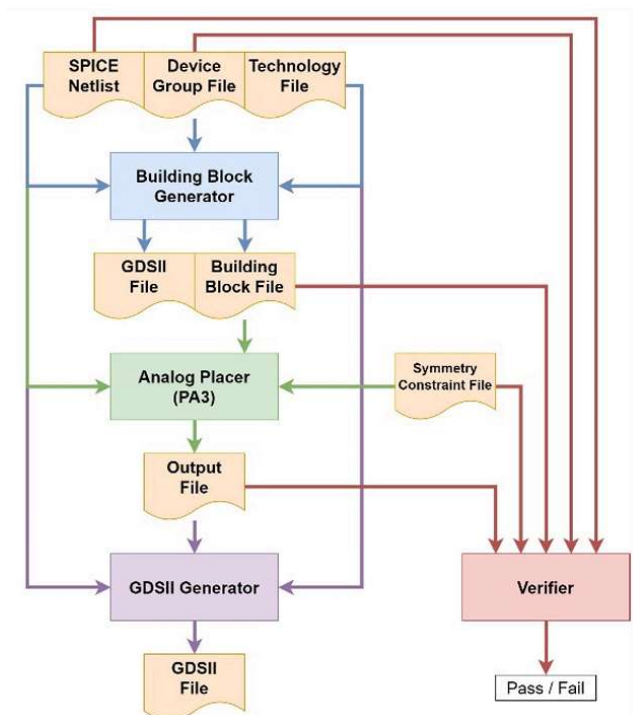
Correctness:

Use Verifier to check the correctness of your placement results. Each test case will be scored independently. Your output must meet the following conditions:

- (1) Contain all devices
- (2) With correct HPWL, area, width, height and building block pattern
- (3) No overlap between any devices
- (4) Meet symmetry constraints

2. How to Compile and Execute and Completion

Work Flowchart



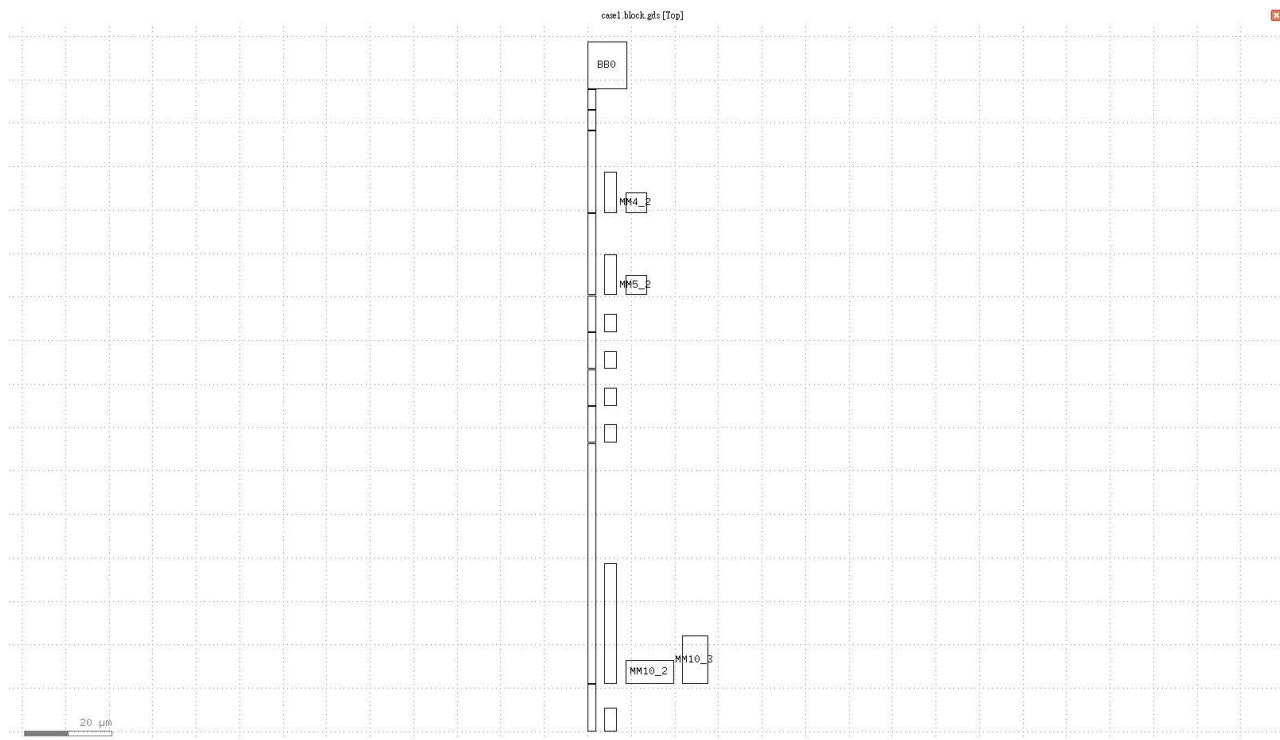
Add a Makefile in folder PA3, choose GNU GCC Compiler in C++ 23 standard.
Upload the files 111521051_PA3.cpp, case1 all inputfile, case2 all inputfile.

Step1. Run BuildingBlockGenerator to generate building blocks

command:

BuildingBlockGenerator <netlist_file> <tech_file> <device_group_file>
<building_block_file>

cas1.block



Step3. Use make to compile the program.

```
(PA3) [s111521051@cad ~/PA3]$ make clean
(PA3) [s111521051@cad ~/PA3]$ ls
111521051_PA3.cpp  case1  case1.block  case1.output  case2  case2.block  case2.output  example  Makefile  test.block
(PA3) [s111521051@cad ~/PA3]$ make all
(PA3) [s111521051@cad ~/PA3]$ ls
111521051  111521051_PA3.o  case1  case1.block  case1.output  case2  case2.block  case2.output  example  Makefile  test.block
```

step4. Execute the analog placer to generate the placement results.

command:

./<student_id> <expected_aspect_ratio> <netlist_file>
<symmetry_constraint_file> <building_block_file> <output_file>

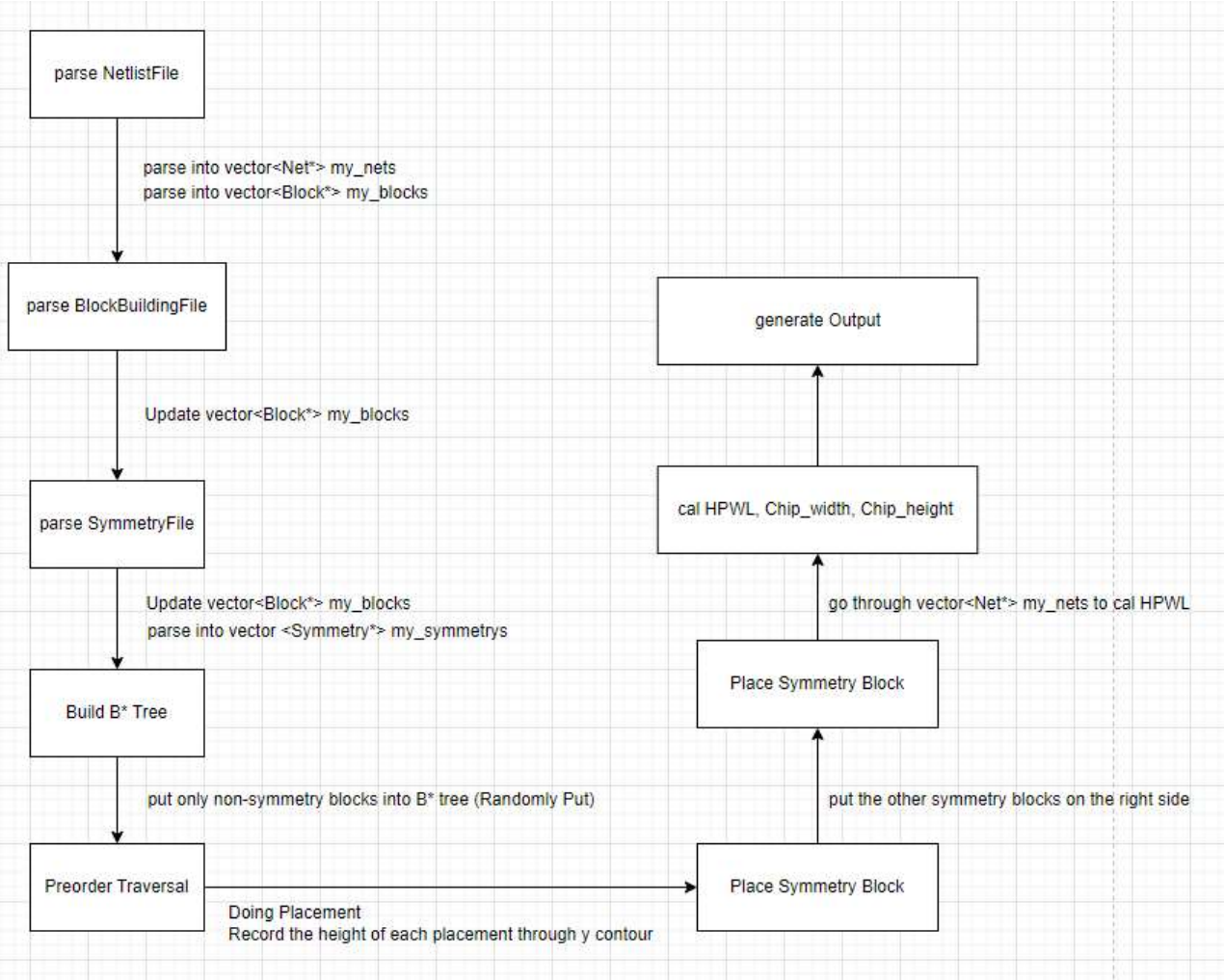
Step5. Use the PA3 Verifier to validations.

Result of case1:

```
(PA3) [s111521051@cad ~/PA3]$ Verifier 3 ./case1/input/case1.netlist ./case1/input/case1.group ./case1/input/case1.sym case1.block case1.output
*****
* Check whether the calculation results of the device info. match your own calculation results or not *
*****
*
*      From Device Info.      From Your Output      *
* 1. Total HPWL      114.705      114.705      *
* 2. Chip Area      549.7272      549.7272      *
* 3. Chip Width      18.93      18.93      *
* 4. Chip Height      29.04      29.04      *
*****
Correctness:      PASS
Meet Symmetry Constraints:      PASS
Cost      276.81653
```

result of case2:

```
(PA3) [s111521051@cad ~/PA3]$ Verifier 3 ./case1/input/case1.netlist ./case1/input/case1.group ./case1/input/case1.sym case1.block case1.output
*****
* Check whether the calculation results of the device info. match your own calculation results or not *
*****
*
*      From Device Info.      From Your Output      *
* 1. Total HPWL      114.705      114.705      *
* 2. Chip Area      549.7272      549.7272      *
* 3. Chip Width      18.93      18.93      *
* 4. Chip Height      29.04      29.04      *
*****
Correctness:      PASS
Meet Symmetry Constraints:      PASS
Cost      276.81653
```



4. Progreem Details (Algorithms and Data Structures)

Read input file to build my_nets, my_blocks, my_symmetrys;

```
class Net {
public:
    string name;
    vector<Block*> blocks;

    Net(){};
    void add_block(Block* block);
};

class Block {
public:
    string name; // BB0
    vector<Net*> nets;
    vector<Dimension*> dimensions; // (width, heighr, col, row)
    int chosenBlock;
    bool isgroupblock = false;
    vector<string> groupedBlocksName; // MM0 MM1
    bool isSymmetry = false;
    double center_x; // for HPWL
    double center_y; // for HPWL
    Block(){};

    void add_net(Net* net);
    void add_nets(const vector<Net*>& newNets);
    void add_dimensions(Dimension* dim);

    int choose_dim_by_area(); // for B* tree
    int choose_dim_by_ratio(); // for B* tree
};

class Symmetry {
public:
    string name;
    double x; //x座標
    vector<Block*> blocks;
    Symmetry(){};
    bool has_two_block = false;
    void add_block(Block* block){
        blocks.push_back(block);
    }
};

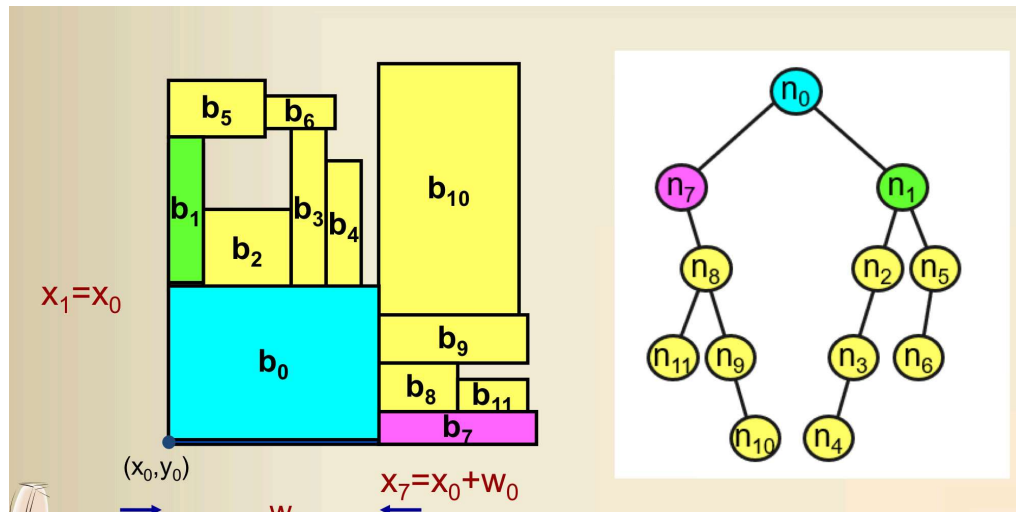
//global variable
vector<Net*> my_nets;
vector<Block*> my_blocks;
vector<Symmetry*> my_symmetrys;
```

Store the parsed data as 3 global variables, while only storing pointers for ease of modification.

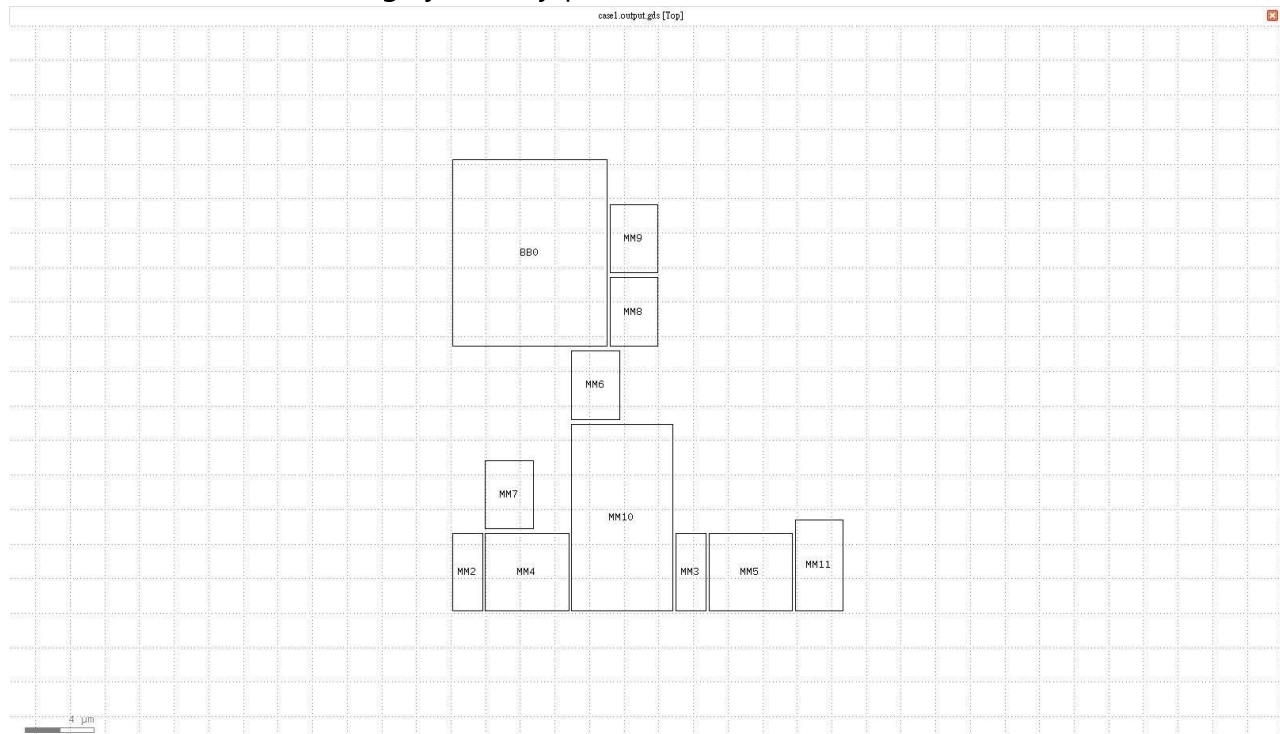
For example, when a symmetry in my_symmetrys is modified to correspond to a block, it can update the content of the block in my_blocks.

B* tree

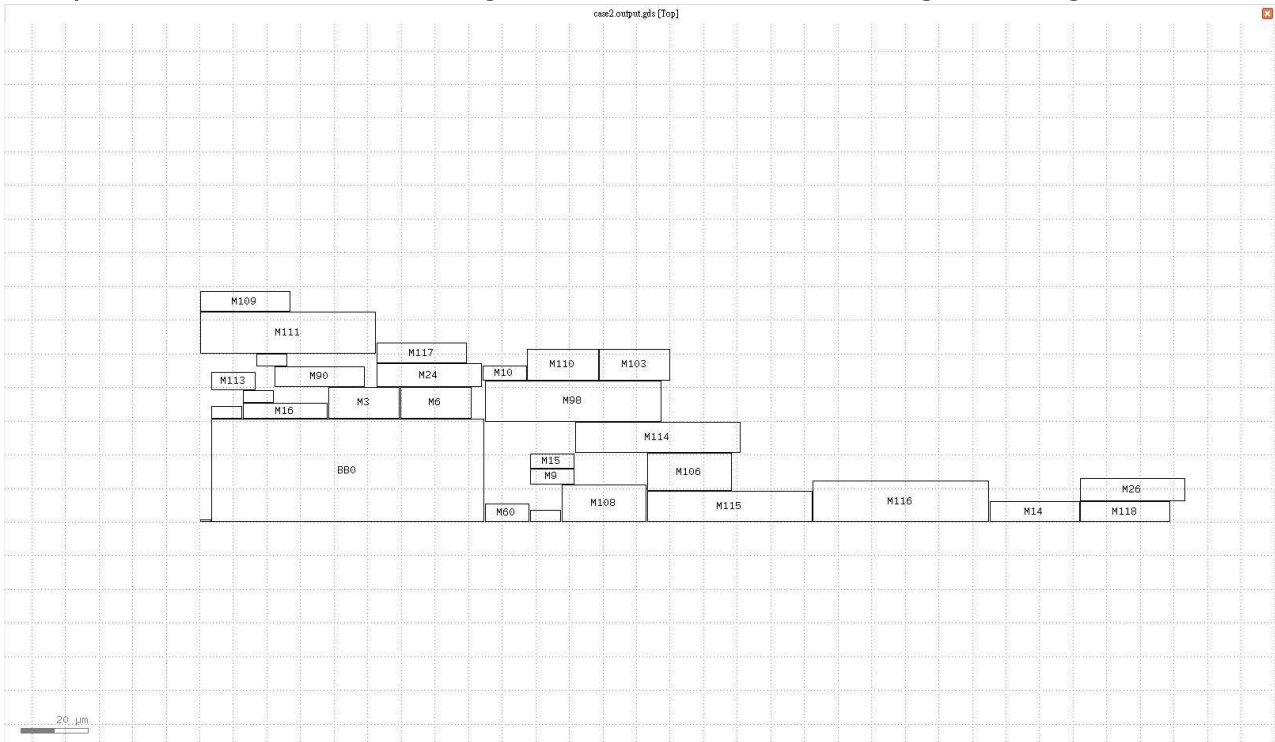
The structure only allows stacking upwards and to the right. If placing an item upwards, it goes to the right child. Instead, if placing to the right, it goes to the left child.



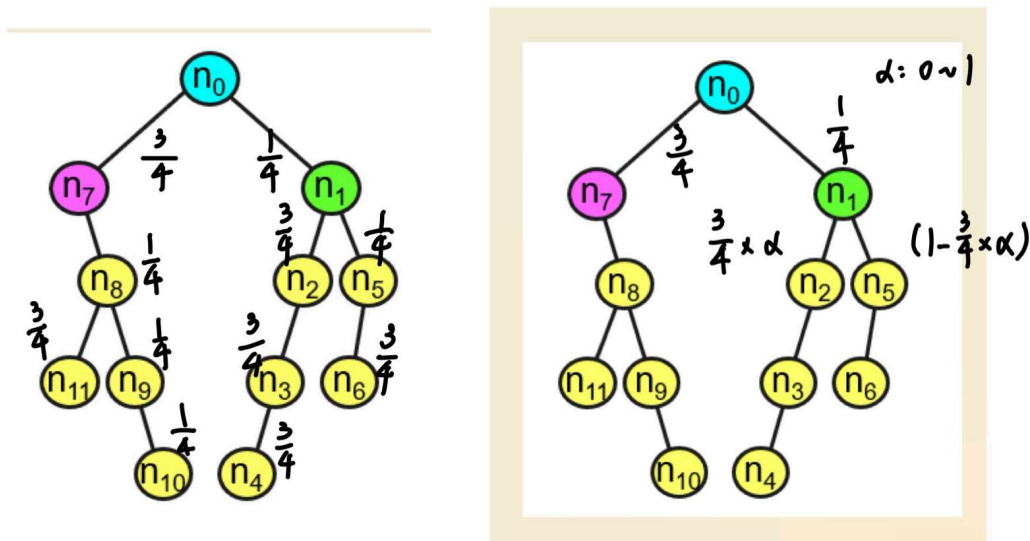
Case1 without considering symmetry placement



Case2 without considering symmetry placement, Clearly, without implementing a decay rate, it would result in a significant bias towards stacking to the right.



On the left side is my design, and on the right side is a way to improve it.



Preorder Traversal

Preorder Traversal Pseudo Code

Use **preorderTraversal** to perform placement. The x-coordinate for placement should be updated based on the parent node's placement result in the tree. For the y-coordinate, maintain a **ycontour** to keep track of it.


```

procedure preorderTraversal(node, place_x):
    if node is null:
        return          // initial condition

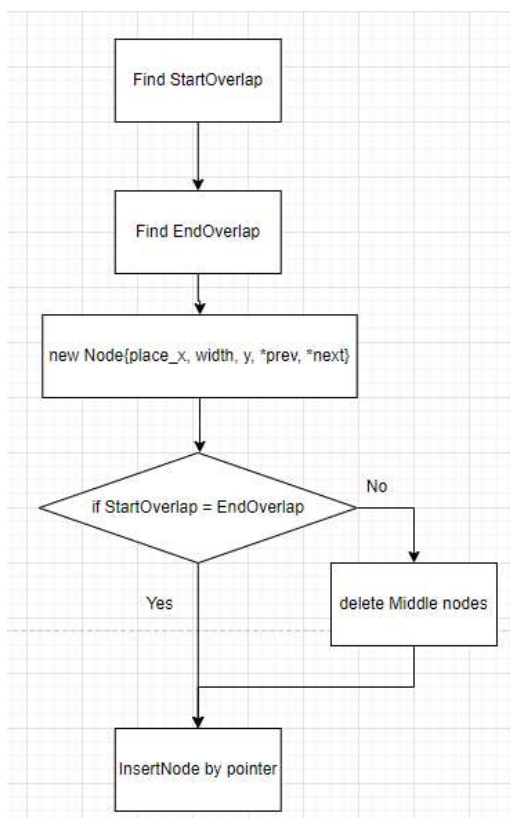
    choose height, width by best ratio.
    calculate Y position for node
    insert node at (place_x, place_y)
    update chip width and height
    record node position in placement
    traverse (left child with updated place_x)    //recursive
    traverse (right child with original place_x)  //recursive

```

Y Contour

I designed a double-linked list structure for the y contour, along with two functions. One function is used to update the height, and the other is to find a y-coordinate for a given x-coordinate from the double-linked list.

InsertNode Flow Chart

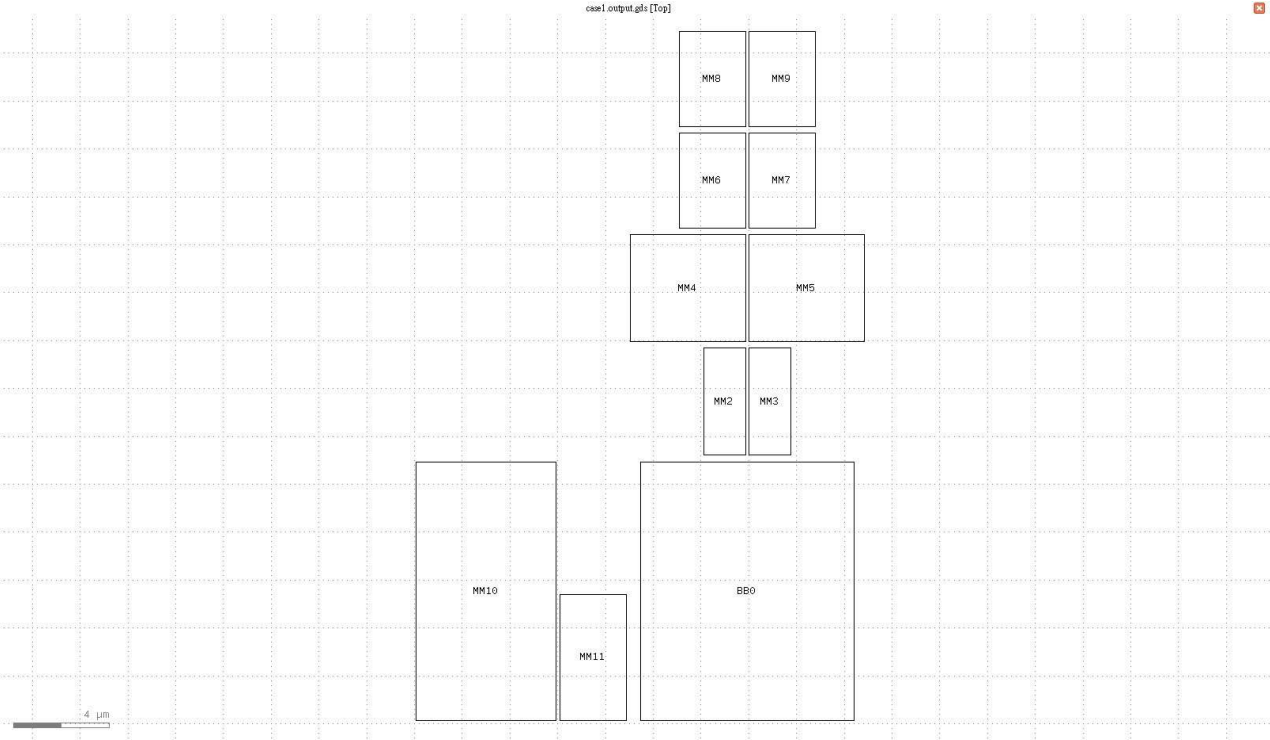


Symmetry Placement

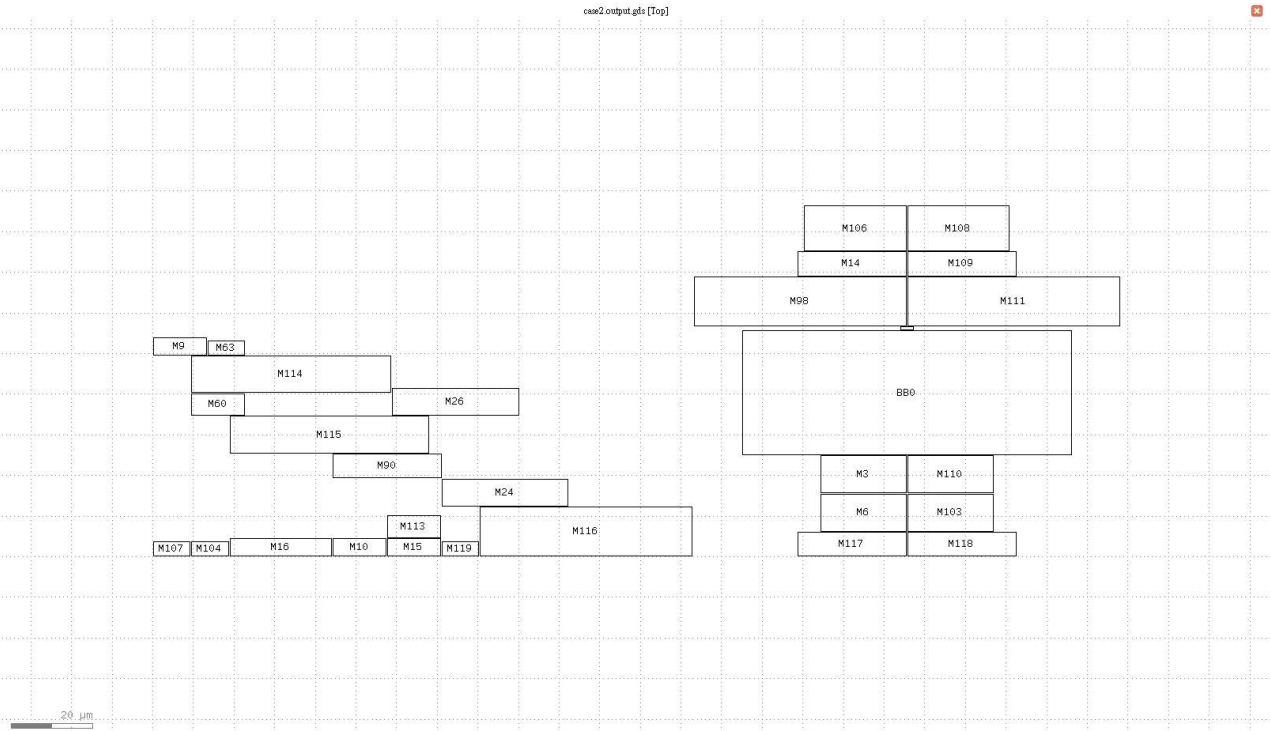
For dealing with symmetry, I'm struggling to come up with a solution. Ultimately, I decided to place all symmetric items on the right side and stack them upwards in sequence.

In cases without symmetry, I implemented the B* tree and preorder traversal as described above.

case1 symmetry placement

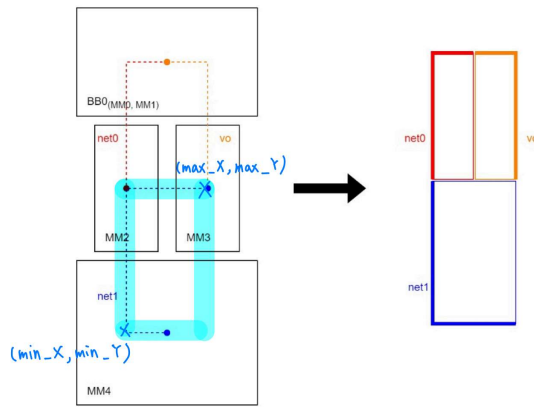


case2 symmetry placement



Calculate HPWL

Go through **my_nets** and find each net's included blocks. Determine the **min_x**, **max_x**, **min_y**, and **max_y** coordinates of these blocks. Finally, compute the total HPWL (Half Perimeter Wire Length) by summing up the HPWL for each net.



5. Perturbation Strategy and Operations & Cost Function

Due to time constraints, only an initial partition has been made. Based on the design of the B* tree, if continued, I intend to implement Simulated Annealing (SA) as follows:

Perturbation: Exchange any two nodes between trees.

Move to Neighbor Possibility: If the new solution is worse, the possibility is calculated as $\exp((\text{old_cost} - \text{new_cost}) / \text{temperature})$. If it's better, the possibility is set to 1.

Cost Function: Cost function is defined as $(0.25 * \text{ChipArea} + 0.75 * \text{TotalHPWL}) * (1 + (\text{AR} - \text{expectedAR}) / 3)^2$, similar to the cost function defined in the problem statement.

6. My Suggestion

The difficulty of Assignment 3 has increased significantly, especially with regards to the design of ycontour. I found it particularly challenging, and discussions with other classmates reveal that no one seems to have been able to complete all the procedures taught in class. The process involves building the B* tree first, creating one half of the symmetric block, copying the other half from the left side. Then, performing placement use preorder traversal, including scoring and optimizing the B* tree. Overall, I feel the difficulty of the assignment is quite high.

While high difficulty is not a major issue, it leads to two types of responses:

1. Some students merely aim to pass the Verifier by hastily conducting placement.
2. Other students manage to partially implement the algorithms discussed in class but get stuck at certain steps.

I suggest increasing the proportion of scores attributed to the demo while lowering the weightage on correctness and quality. This adjustment would at least ensure that the scores of the second type of students are not lower than those of the first type.

In conclusion, this assignment was indeed very challenging, but it proved to be highly beneficial. In particular, designing ycontour has made me much more proficient in manipulating pointers.